UPRISE – An Intelligent AI-Assisted Trading Simulation System

Nandan Khatod
Department of Information Technology,
K J Somaiya Institute of Technology
Mumbai, India
nandan.khatod@somaiya.edu

Omkar Kulkarni
Department of Information Technology,
K J Somaiya Institute of Technology
Mumbai, India
omkar.kk@somaiya.edu

Bhumi Khona
Department of Information Technology
K J Somaiya Institute of Technology
Mumbai, India
bhumi.kk@somaiya.edu

Dr. Seema Yadav
Professor
Department of Information Technology,
K J Somaiya Institute of Technology
Mumbai, India
syadav@somaiya.edu

Prof. Raadhika Kotecha
Head of Department
Department of Information Technology,
K J Somaiya Institute of Technology
Mumbai, India
radhika.kotecha@somaiya.edu

Abstract*— The Intelligent AI-Assisted Trading Simulation System is a financial technology platform designed to provide a realistic and risk-free environment for learning and experimenting with stock trading. The system integrates a C++ Limit Order Book (LOB) matching engine using price-time priority with a React frontend, Node.js/Express backend, MongoDB, WebSocket communication, and a Python-based machine-learning service. Historical market data are processed using technical indicators such as Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), and Moving Averages. The machine-learning module uses LSTM as the primary model for short-term market prediction, with Ridge Regression and Random Forest used as baseline models for performance comparison. The platform combines AI-assisted analysis, order execution, portfolio tracking, and real-time updates, providing a modular environment for evaluating trading strategies without real-money transactions.

Index Terms*—Artificial Intelligence, Paper Trading, Limit Order Book, Price-Time Priority, LSTM, Machine Learning, Algorithmic Trading, WebSockets, Financial Technology.

I. INTRODUCTION

Financial markets generate large volumes of sequential data containing price, volume, and volatility information. Machine-learning techniques enable intelligent financial analysis and algorithmic trading, but real-market experimentation involves financial risk and complex execution mechanisms.

Traditional paper-trading platforms provide risk-free practice but may simplify market-microstructure concepts such as order queues, price-time priority, partial execution, and cancellation. Limit Order Book research highlights the importance of such information for short-term forecasting [1]–[3].

This work presents an Intelligent AI-Assisted Trading Simulation System that integrates AI-based market prediction with a C++ Limit Order Book matching engine. The system employs LSTM for short-term market prediction, while Ridge Regression and Random Forest are evaluated as baseline models for comparison with technical indicators, portfolio management, order execution, and real-time updates, providing a risk-free environment for trading experimentation.

The main contributions are:

1. An integrated AI-assisted paper-trading platform.
2. A C++ Limit Order Book with price-time priority and simulated execution.
3. LSTM, Ridge Regression, and Random Forest models for market prediction.
4. Technical feature engineering using SMA, RSI, MACD, returns, and volatility.
5. Integrated portfolio, P&L, order, and transaction management.

II. RELATED WORK

Research in intelligent trading systems can broadly be divided into financial time-series forecasting, machine-learning-based trading, limit-order-book modelling, and algorithmic execution.

Zhang et al. proposed DeepLOB, which uses deep-learning architectures to extract spatial and temporal patterns from limit-order-book data for short-term market prediction [1]. Ntakaris et al. introduced a benchmark dataset and

evaluation framework for mid-price forecasting from limit-order-book information [2]. Tsantekidis et al. investigated convolutional neural networks for forecasting stock prices using order-book information [3].

Deep-learning approaches such as LSTM have also been widely investigated for financial time-series forecasting because recurrent architectures can model sequential dependencies. Unlike conventional regression approaches, recurrent networks can process sequences of observations and retain information from previous time steps.

Traditional machine-learning models such as linear regression, Ridge Regression, and Random Forest are useful as baseline approaches because they provide comparatively simpler models for financial prediction. Such models also allow the performance of more complex sequential architectures to be evaluated against conventional techniques.

Existing research frequently focuses on either prediction accuracy or trading strategy performance. Similarly, research on limit-order books often concentrates on market prediction rather than educational trading simulation. The proposed work combines **AI-assisted prediction, realistic simulated order matching, portfolio management, visualization, and real-time interaction** within a single deployable platform.

III. PROBLEM STATEMENT

Learning financial trading through real markets exposes beginners to significant financial risk, while simplified simulation platforms may not adequately represent the mechanisms involved in order-driven markets.

The problem addressed in this work is the development of an integrated simulation environment that can provide:

1. Risk-free trading experimentation.
2. AI-assisted financial analysis.
3. Historical market-data processing.
4. Short-term price prediction.
5. Technical-indicator-based trading suggestions.
6. Realistic order matching using a Limit Order Book.
7. Portfolio and P&L tracking.
8. Interactive market visualization.
9. Real-time system updates.

The proposed platform addresses these requirements through a modular architecture combining web technologies, machine learning, persistent storage, real-time communication, and a dedicated C++ matching engine.

IV. PROPOSED SYSTEM

The proposed system consists of major components:

1. **Frontend Interface**
2. **Backend API Layer**
3. **Machine-Learning Service**
4. **Matching Engine**
5. **Database Layer**
6. **Real-Time Communication Layer**

The overall system can be represented as:

User → React Interface → Node.js Backend → ML / Matching Engine → Database → Real-Time Dashboard

The React frontend provides market visualization, stock selection, order entry, portfolio information, and dashboards. The Node.js and Express.js backend manages API requests, authentication, user information, trading requests, and communication with other services.

The Python machine-learning service processes historical market data, calculates technical indicators, performs model inference, and generates prediction outputs. Trading orders are transmitted to the C++ matching engine, where they are processed using a Limit Order Book and price-time priority. MongoDB provides persistent storage for user information, orders, trades, positions, market information, and related trading records.

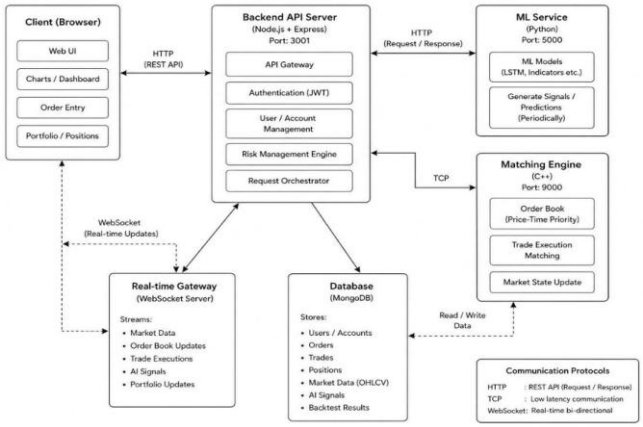


Fig 1. Proposed System Architecture

V. SYSTEM WORKFLOW

The complete workflow is divided into the following stages.

A. User Interaction

The user accesses the web-based trading interface and selects a stock, views market information, examines technical indicators, or submits a trading order.

B. Market Data Acquisition

Historical and market information is obtained through the Yahoo Finance/yfinance interface. The system supports Indian stock symbols and displays prices in Indian Rupees.

C. Feature Engineering

The raw market data are processed to generate technical features including:

- Closing price
- Volume

- SMA-5
- SMA-20
- SMA-50
- RSI
- MACD
- MACD signal
- Returns
- Rolling volatility
- Volume-related features

D. AI Prediction

The processed information is supplied to the machine-learning module. LSTM processes sequential observations, while Ridge Regression and Random Forest provide alternative prediction models.

E. Trading Decision Support

Prediction results are interpreted into trend and expected-movement information. These outputs provide decision support to the user.

F. Order Submission

When the user submits a trading order, the request is forwarded through the backend to the matching engine.

G. Order Matching

The C++ engine maintains the order book for each symbol and applies price-time priority to determine execution.

H. Portfolio Update

Executed trades update the user's simulated positions, portfolio value, P&L, and transaction history.

I. Real-Time Update

WebSocket communication provides live updates to the frontend for relevant market and trading events.

VI. SYSTEM ARCHITECTURE

The proposed workflow begins with a user interaction such as selecting a stock, requesting an AI prediction, or placing a trading order. The React frontend communicates with the Node.js and Express.js backend through REST APIs. Based on the requested operation, prediction requests are routed to the Python-based machine-learning service, which processes historical market data and technical indicators before returning the prediction output. Trading orders are forwarded to the C++17 matching engine, where a symbol-wise Limit Order Book applies price-time priority to determine order execution, including partial fills and cancellations. Order and trade information is persisted through the backend using MongoDB, while WebSocket communication provides real-time updates to the frontend dashboard. This architecture separates user interaction, business logic, machine-learning processing, data persistence, and trade execution into modular components.

VII. MARKET DATA AND FEATURE ENGINEERING

Historical market information forms the primary input to the machine-learning module. The system uses OHLCV market information together with derived technical indicators.

MARKET FEATURES USED BY THE SYSTEM

TABLE I

Category	Features
Price	Open, High, Low, Close
Volume	Trading Volume
Moving Average	SMA-5, SMA-20, SMA-50
Momentum	RSI-14
Trend	MACD
MACD	MACD Signal
Returns	1-day, 5-day, 20-day
Volatility	Rolling volatility
Volume Analysis	Volume / 20-day average

The implementation calculates additional indicators including Bollinger Bands, although the primary LSTM feature vector uses closing price, volume, SMA-5, SMA-20, RSI, and MACD.

VIII. LSTM MODEL

LSTM is selected because financial data form a time sequence in which previous observations can provide information about subsequent observations.

The proposed LSTM architecture consists of:

Input Sequence → LSTM(50) → Dropout(0.2) → LSTM(50) → Dropout(0.2) → Dense(25) → Dense(1)

The model uses a sequence length of 60 observations.

The LSTM prediction can be represented as:

$$\hat{y}_{t+1} = f(X_{t-59}, X_{t-58}, .., .. X_t)$$

where X_t represents the feature vector at time t, and $\hat{y}_{t+1}$represents the predicted subsequent value.

The model uses the Adam optimizer and Mean Squared Error (MSE) as the loss function.

IX. RIDGE REGRESSION

Ridge Regression provides a regularized linear baseline for comparison with the nonlinear and sequential models.

The Ridge objective function is:

$$J(\beta) = \sum_{i=1}^{N} (y_i - \beta^T x_i)^2 + \alpha \sum_{j=1}^{p} \beta_j^2$$

where:

- $x_i$is the input feature vector,

- y_i is the target,
- β represents model coefficients,
- α controls regularization strength.

The implemented configuration uses:

X. RANDOM FOREST

Random Forest is used as a nonlinear tree-based baseline. It combines predictions from multiple decision trees to model nonlinear relationships between market features and the prediction target.

The implemented Random Forest configuration contains:

100 trees

Maximum depth = 10

Random state = 42

The final prediction is obtained by aggregating the predictions generated by the individual trees.

XI. TRAINING CONFIGURATION

TABLE II

Parameter	LSTM	Ridge	Random Forest
Sequence Length	60	—	—
LSTM Units	50 + 50	—	—
Dropout	0.2	—	—
Dense Layer	25	—	—
Optimizer	Adam	—	—
Loss	MSE	—	—
Maximum Epochs	50	—	—
Batch Size	32	—	—
Validation Split	20%	—	—
Regularization α	—	1.0	—
Trees	—	—	100
Maximum Depth	—	—	10
Random State	—	—	42

These parameters correspond to the model configuration used in the project implementation.

XII. AI PREDICTION AND TRADING SUGGESTION

The prediction service processes the latest available market information and generates an expected market movement.

The prediction output is interpreted into:

- Predicted value
- Expected movement
- Trend
- Confidence

The decision layer classifies the prediction into directional categories such as bullish, bearish, or neutral depending on the generated movement and confidence.

The AI output is intended as **decision support**. It is presented to users alongside market information and technical indicators rather than being treated as a guaranteed trading signal.

XIII. ORDER MATCHING ENGINE

The trading engine is based on a Limit Order Book.

A separate order book is maintained for each trading symbol. Orders are organized according to price-time priority.

For buy orders, higher prices receive priority. For sell orders, lower prices receive priority. When two orders have the same price, the order that arrived earlier receives priority.

For a compatible buy and sell order:

$$Q_{executed} = \min(Q_{buy}, Q_{sell})$$

This mechanism supports partial fills when the available quantities of the two orders differ.

The engine supports:

- Market orders
- Limit orders
- Price priority
- Time priority
- Partial fills
- Order cancellation
- Order status management

This provides a more realistic simulation of order-driven execution than directly updating a portfolio after every submitted order.

XIV. ORDER EXECUTION

For a limit buy order, execution occurs when a compatible sell order exists at a suitable price. Similarly, a sell order can execute when a compatible buy order is available.

For example, if a buy order requests 100 shares while a compatible sell order contains 60 shares:

$$Q_{executed} = \min(100, 60) = 60$$

The buy order therefore retains 40 shares as its remaining quantity.

This mechanism allows the simulation to represent partial execution and order queues.

XV. SECURITY AND REAL-TIME COMMUNICATION

The system uses JWT-based authentication and HTTP-only cookies for session management. User-specific trading resources are isolated through authenticated backend requests.

The real-time communication layer uses WebSockets to provide live market and system updates without requiring repeated page refreshes.

REST APIs provide standard request-response communication, while TCP sockets connect the backend with the C++ matching engine.

The security architecture considers:

- Authentication
- Authorization
- Session protection
- Input validation
- User ownership
- Protected trading resources

XVI. IMPLEMENTATION TECHNOLOGIES

TABLE III
IMPLEMENTATION TECHNOLOGIES

Component	Technology
Frontend	React.js
Backend	Node.js, Express.js
Authentication	JWT, HTTP-only Cookies
Database	MongoDB
ML Service	Python, Flask
Deep Learning	TensorFlow
ML Models	LSTM
Market Data	Yahoo Finance / yfinance
Matching Engine	C++17

XVII. EVALUATION METRICS

The system is evaluated at three levels: machine-learning prediction, trading simulation, and order-execution performance.

A. Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N} | y_i - \hat{y}_i |$$

B. Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

C. Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

D. Mean Absolute Percentage Error

$$MAPE = \frac{100}{N}\sum_{i=1}^{N} \left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

E. Directional Accuracy

$$DA = \frac{\text{Correctly predicted directions}}{\text{Total predictions}} \times 100$$

F. Trading Return

$$Return = \frac{V_f - V_i}{V_i} \times 100$$

where V_i is the initial portfolio value and V_f is the final portfolio value.

G. Maximum Drawdown

$$MDD = \max_t \left(\frac{Peak_t - Value_t}{Peak_t}\right) \times 100$$

H. Matching Throughput

$$Throughput = \frac{\text{Orders processed}}{\text{Execution time}}$$

These metrics provide complementary information about prediction quality, financial simulation, and execution performance.

XVIII. DISCUSSION

The proposed system combines financial prediction and simulated market execution within a single architecture. Unlike a prediction-only system, the platform allows prediction outputs to be viewed alongside market information and subsequently used during simulated trading.

The use of multiple prediction models provides a comparative framework for evaluating different modelling approaches. LSTM is designed to capture sequential relationships in historical observations, while Ridge Regression and Random Forest provide conventional baselines.

The Limit Order Book provides an additional level of realism by representing order queues and price-time priority. Partial fills and cancellation further improve the representation of order-driven execution.

The combination of machine learning, order matching, portfolio tracking, and real-time visualization creates an integrated educational environment for studying both the analytical and execution aspects of trading.

XIX. LIMITATIONS

The platform is designed as a simulated trading environment and therefore does not execute real-money transactions or directly connect users to live exchanges.

Historical market data cannot capture every factor influencing financial markets. Events such as breaking news, macroeconomic conditions, company announcements, investor sentiment, and unexpected market events may influence prices beyond the features used by the prediction models.

The simulated matching engine also cannot reproduce every characteristic of a production exchange, including actual network conditions, transaction costs, regulatory mechanisms, market impact, slippage, and the full complexity of market microstructure.

Machine-learning performance may also vary across securities, market regimes, data frequencies, and evaluation periods.

XX. FUTURE SCOPE

Future work can extend the platform by evaluating advanced models such as **GRU, CNN-LSTM, and Transformers**, along with financial news and sentiment analysis. Reinforcement learning can also be explored for automated trading strategies.

The system can further incorporate advanced risk-management features such as **stop-loss, take-profit, position limits, and risk-adjusted portfolio optimization**. The matching engine can be extended with additional order types, transaction costs, slippage, and market-impact modelling. Future improvements may also include **model-drift detection, automated retraining, cloud deployment, and scalable multi-user simulation**.

XXI. CONCLUSION

This paper presented an **Intelligent AI-Assisted Trading Simulation System** that integrates artificial intelligence with realistic simulated trading. The system combines a React frontend, Node.js/Express backend, MongoDB, Python-based machine learning, WebSocket communication, and a C++ Limit Order Book matching engine.

The ML module uses **LSTM, Ridge Regression, and Random Forest** models with historical market data and technical indicators. The trading platform supports order management, portfolio tracking, P&L calculation, and transaction history, while the matching engine implements **price-time priority, market and limit orders, partial fills, and cancellations**.

The proposed system provides a unified, risk-free environment for studying financial prediction, paper trading, and market microstructure. Its modular architecture also provides a foundation for future enhancements such as advanced deep-learning models, sentiment analysis, reinforcement learning, improved risk management, and more detailed market simulation.

REFERENCES

[1] Z. Zhang, S. Zohren, and S. J. Roberts, "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books," *IEEE Transactions on Signal Processing*, vol. 67, no. 11, pp. 3001–3012, Nov. 2019.

[2] A. Ntakaris, G. Iosifidis, S. Zohren, and M. Kanniainen, "Benchmark Dataset for Mid-Price Forecasting of Limit Order Book Data with Machine Learning Methods," *International Journal of Forecasting*, vol. 37, no. 8, pp. 852–866, Dec. 2018.

[3] A. Tsantekidis, G. Passalis, D. Tzelepis, et al., "Forecasting Stock Prices from the Limit Order Book Using Convolutional Neural Networks," in *Proc. IEEE 19th Int. Conf. on Business Informatics*, 2017, pp. 125–132.

[4] "Predicting Stock Market Trends Using Long Short-Term Memory (LSTM) Networks: A Deep Learning Approach for Financial Time-Series Forecasting," *IEEE Xplore*, 2025.

[5] "Time Series Forecasting in Financial Market: Long Short-Term Memory (LSTM) Approach for Stock Price Prediction," *IEEE Xplore*, 2025.

[6] Y. Xiao, M. Chen, H. Li, et al., "LiT: Limit Order Book Transformer for Short-Term Market Predictions," *Frontiers in Artificial Intelligence*, 2025.

[7] Y. Wu, M. Mahfouz, D. Magazzeni, and M. Veloso, "Towards Robust Representation of Limit Order Books for Deep Learning Models," *arXiv preprint*, 2021.

[8] A. Ntakaris, A. Tefas, J. Kanniainen, et al., "Feature Engineering for Mid-Price Prediction with Deep Learning," *arXiv preprint*, 2019.

[9] "Forecasting and Prediction Using Deep Learning in Financial Markets – CNN-LSTM Approach," *IEEE Xplore*, 2025.

[10] K. Brown and A. Johnson, "Optimization Techniques for WebSocket-Based Real-Time Communication Systems," *IEEE Transactions on Network and Service Management*, 2024.